



《数据库系统》课内实验指导书

——实验 3：存储过程、函数与触发器

一、实验目的

- 1、学会存储过程的创建、执行和删除。
- 2、学会函数的创建、执行和删除。
- 3、学会触发器的创建、执行和删除。

二、实验要求

- 1、在部署的 openGauss 数据库系统（通过 docker 或华为云）完成存储过程、函数和触发器的练习；
- 2、第一阶段在交互式 gsql 上完成 SQL 语句测试，第二阶段在 jupyter notebook 上完成展示；
- 3、提交内容：一份 txt 文本（包含题目、SQL 语句）+一份 notebook 文件（包含 SQL 语句的执行过程）+一份 word/markdown 实验报告（包含题目、SQL 语句、必要截图和文字说明）；
- 4、提交时间：下周日（4.6）晚上之前；
- 5、整合在一个压缩包里，文件命名为：姓名-班级-学号-实验 3。

三、实验内容

方案一：完成指定的存储过程、函数和触发器题目的练习。

方案二：自行设计 15 条 SQL 语句的练习，要求覆盖存储过程、函数和触发器的使用。

方案一实验题目：

一、存储过程：

1) 创建并执行一个带输入参数的存储过程 `proc_product1`，通过该存储过程可以根据输入参数员工编号进行员工具体信息的查询：包括名字、生日、性别、在不



同时间段下的工资。

2) 创建并执行一个带输入参数和输出参数的存储过程 `proc_product2`，通过该存储过程可以根据输入参数员工编号查询出该员工具体信息：包括名字、生日、性别、在不同时间段下的工资。

3) 删除存储过程 `proc_product2`。

二、函数：

1) 创建一个函数，实现根据输入的 `dept_no`(员工编号)查看该员工所属部门(`department`)、职位(`title`)和工资(`salary`)。

2) 创建一个函数 `INSERT_Func`，向 `departments` 表中插入数据（部门编号、部门名称），并将 `departments` 中的部门数作为返回值。

3) 创建一个触发器函数，实现向 `departments` 表中插入数据。

4) 创建并执行一个函数 `FUNC1`，通过该函数可以根据输入参数员工编号查询出该员工具体信息：包括名字、生日、性别、在不同时间段下的工资（放在数组中）。

5) 创建并执行一个函数 `FUNC2`，基于 `FUNC1`，通过该函数可以根据输入参数员工编号查询出该员工具体信息：包括名字、生日、性别、在不同时间段下的工资的均值。（函数嵌套）

三、触发器：

1) 创建一个触发器，实现当删除 `employees` 表的一条员工记录时，将该数据也同步删除在 `dept_manager`、`dept_emp`、`salaries`、`titles` 表中的记录。

2) 在 `salaries` 表上创建一个 `BEFORE` 行级触发器（名称：`INSERT_OR_UPDATE_SAL`）以实现如下完整性规则：员工的工资不得低于 80000 元，如果低于 80000 元，自动改为 80000 元。

3) 验证触发器是否正常工作：分别执行以下 A,B 两种操作，验证 `INSERT_OR_UPDATE_SAL` 触发器是否被触发？工作是否正确？如果正确，请观察 `salaries` 表中数据的变化是否与预期一致。

插入两条新数据 (`10005,80001,'1995-06-24 00:00:00','1996-06-24 00:00:00'`),(`10006,78888,'2000-06-22 00:00:00','2002-09-22 00:00:00'`);

更新数据:将 `id` 为 10001 的员工在所有时期的工资改为 90000。

4) 删除触发器 `INSERT_OR_UPDATE_SAL`。

附录

1 创建和调用存储过程

1.1 创建

```
CREATE PROCEDURE <存储过程名> ([参数模式] 参数名 数据类型 [, ...])
AS
BEGIN
    <SQL 语句块>
END;
/
```

注：

- 括号内指定参数列表（也可以没有参数，但括号不可省略）
- BEGIN…END 之间是该存储过程要执行的 SQL 语句
- 终止符标志着存储过程的结束
- 参数模式为： [IN|OUT|INOUT] 参数名 参数数据类型
 - IN 表示该参数为输入参数，OUT 表示该参数为输出参数，INOUT 表示该参数既可以输入也可以输出
 - 参数名指定该参数的名称
 - 参数数据类型指定该参数的数据类型，比如 INT, FLOAT, VARCHAR(n), DATE 等

1.2 调用

```
CALL <存储过程名()>;
```

示例：

首先，创建 sc 表，并加入两条学生记录（‘1’，‘12’，101）和（‘001’，‘1’，105）。

```
CREATE TABLE sc (
    sno CHAR(8),      -- 学生编号
    cno CHAR(4),      -- 课程编号
    grade FLOAT       -- 成绩
);
```

```
);  
INSERT INTO sc VALUES ('1','12',101),('001','1',105);
```

【例 1】创建名为 PROC_COURSE_AVG_GRADE 的存储过程，计算某门课程的平均学生成绩，并将结果存储在某个变量中。

1.创建存储过程

```
CREATE PROCEDURE PROC_COURSE_AVG_GRADE (IN    course_id  
CHAR(4), OUT avg_grade FLOAT)  
AS  
BEGIN  
    SELECT AVG(grade) INTO avg_grade FROM sc WHERE cno = course_id;  
END;  
/
```

2.测试：查询 1 号课程的平均学生成绩，并将结果存储在 avg_grade 变量中，并输出 avg_grade 变量

```
CALL PROC_COURSE_AVG_GRADE ('1', avg_grade);
```

【例 2】创建名为 PROC_COURSE_GRADE 的存储过程，输出某门课程的学生信息。

方案一：使用存储过程 + 游标返回结果集

1.创建存储过程

```
CREATE PROCEDURE PROC_COURSE_INFOR (  
    IN    course_id CHAR(4),          -- 输入参数：课程编号  
    INOUT result_cursor REFCURSOR -- 输出参数：结果集游标  
)  
AS  
BEGIN  
    OPEN result_cursor FOR  
        SELECT * FROM sc WHERE cno = course_id;    -- 查询指定课程的  
        成绩记录  
END;  
/
```



2.测试：使用 CALL 语句调用存储过程。并用 FETCH 获取游标，查询 1 号课程的学生信息

BEGIN;

CALL PROC_COURSE_INFOR('1','mycursor'); -- 调用存储过程

FETCH ALL FROM mycursor; -- 获取游标中的全部结果

COMMIT;

方案二：通过函数直接返回结构化数据

1.创建存储过程

CREATE FUNCTION FUNC_COURSE_INFOR (course_id CHAR(4))

RETURNS TABLE (sno CHAR(8), cno CHAR(4), grade FLOAT)

AS \$\$

BEGIN

 RETURN QUERY

 SELECT sc.sno, sc.cno, sc.grade

 FROM sc

 WHERE sc.cno = course_id;

END;

\$\$ LANGUAGE PLPGSQL;

2.测试：查询 1 号课程的学生信息

SELECT * FROM FUNC_COURSE_INFOR('1');

2 创建并调用函数

CREATE [OR REPLACE] FUNCTION <函数名> ([参数名 数据类型 [, ...]])

RETURNS <返回类型>

AS \$\$

[DECLARE

 变量声明]

BEGIN

 <SQL 或 PL/pgSQL 语句>

 RETURN <返回值>;

END;

\$\$ LANGUAGE <语言类型> [选项];

- 创建函数的语法格式为：Returns 返回数据类型；Return 返回数据值。
- 参数的格式与存储过程相似，但是只能是 IN 参数
- RETURNS 语句指定函数返回数据的类型
- openGauss 的语言类型为 PLPGSQL

示例：

【例 3】创建函数，返回某名学生选修的某门课程的成绩。

1.创建函数

```
CREATE FUNCTION FUNC_STU_COU_GRADE(student_id CHAR(10),
course_id CHAR(4))
RETURNS FLOAT
AS $$
DECLARE
result_grade FLOAT;
BEGIN
SELECT Grade INTO result_grade FROM SC WHERE Sno = student_id AND Cno
= course_id;
RETURN result_grade;
END;
$$ LANGUAGE PLPGSQL;
```

2.测试：查询学号 001 的学生 1 号课程成绩。

```
SELECT FUNC_STU_COU_GRADE( '001' , '1' );
```

3 变量

- 可以在存储过程和函数中声明并使用变量
- 变量的作用范围是在 BEGIN…END 语句块中

3.1 变量定义

```
DECLARE <变量名>[, <变量名>, ..., <变量名>] <数据类型> [DEFAULT <默认值>];
```

- 变量名（列表）指定了定义的变量的名称
- 数据类型定义了这些变量的数据类型
- 当指定 DEFAULT 选项时，默认值为这些变量提供了一个默认值。如果未指定 DEFAULT 选项，变量的初始值为 NULL

3.2 变量赋值

```
<变量名>: = <表达式>, [<变量名>: = <表达式>, ..., <变量名>: = <表达式>];
```

或

```
SET <变量名> TO <表达式>;
```

4 流量控制

4.1 IF 语句

```
IF <表达式> THEN <SQL 语句块>;  
[ELSEIF <表达式> THEN <SQL 语句块>;  
...  
ELSEIF <表达式> THEN <SQL 语句块>;]  
ELSE <SQL 语句块>;  
END IF;
```

示例：

【例 4】创建一个名为 FUNC_GRADE_LEVEL 的成绩等级评定函数，参数为学生学号与课程号，返回值为学生该门课程的成绩等级，包括:A[90-100],B[80-90],C[70-80],D[60-70],E[60 以下]。

1.创建函数



```
CREATE FUNCTION FUNC_GRADE_LEVEL(student_id CHAR(10), course_id CHAR(4))
RETURNS VARCHAR(5) AS $$
DECLARE
    level VARCHAR(5) := 'E';
    grade INT;
BEGIN
    SELECT Grade INTO grade FROM SC WHERE Sno = student_id AND Cno = course_id;
    IF grade >= 90 THEN    level := 'A';
    ELSIF grade >= 80 THEN    level := 'B';
    ELSIF grade >= 70 THEN    level := 'C';
    ELSIF grade >= 60 THEN    level := 'D';
    ELSE    level := 'E';
    END IF;
    RETURN level;
END;
$$ LANGUAGE PLPGSQL;
```

2.测试：查询学号 001 的学生的 1 号课程成绩。

```
SELECT FUNC_GRADE_LEVEL('001','1');
```

4.2 LOOP 语句

```
[<标签>:] LOOP <SQL 语句块> END LOOP [<标签>];
```

- 程序执行时，会重复执行 LOOP 后的语句块，直到循环被退出。
- 在 LOOP 语句中，使用 EXIT WHEN 子句可跳出循环。
- LOOP 语句中必须包含 EXIT WHEN 子句，否则会陷入死循环。
- 标签可以用来标志一个 LOOP 语句，为可选项。

示例：

【例 5】创建一个名为 FUNC_SUM1 的函数，参数为 num，计算 $1+2+\dots+(num-1)+num$ 的结果。



1.创建函数

```
CREATE FUNCTION FUNC_SUM1(num INT)
```

```
RETURNS INT
```

```
AS $$
```

```
DECLARE
```

```
    total INT := 0;
```

```
    cur_num INT := num;
```

```
BEGIN
```

```
    LOOP
```

```
        total := total + cur_num;
```

```
        cur_num := cur_num - 1;
```

```
        EXIT WHEN cur_num <= 0;
```

```
    END LOOP;
```

```
    RETURN total;
```

```
END;
```

```
$$ LANGUAGE PLPGSQL;
```

2.测试：调用 FUNC_SUM1 函数，参数为 10。

```
SELECT FUNC_SUM1(10);
```

4.3 WHILE 语句

```
[<标签>:] WHILE <表达式> LOOP  
<SQL 语句块>  
END LOOP;
```

示例：

【例 6】创建一个名为 FUNC_SUM2 的函数，参数为 num，计算 $1+2+\cdots+(num-1)+num$ 的结果。

1.创建函数

```
CREATE OR REPLACE FUNCTION FUNC_SUM2(num INT)
```

```
RETURNS INT
AS $$
DECLARE                                -- 定义局部变量
    total INT := 0;
    cur_num INT := num;
BEGIN                                  -- 定义 WHILE 循环的函数体
    WHILE cur_num > 0 LOOP
        total := total + cur_num;
        cur_num := cur_num - 1;
    END LOOP;
    RETURN total;
END;
$$ LANGUAGE PLPGSQL;
2.测试：调用 FUNC_SUM2 函数，参数为 10。
SELECT FUNC_SUM2(10);
```

5 删除存储过程与函数

```
DROP PROCEDURE <存储过程名>;
```

```
DROP FUNCTION <函数名>;
```

6 创建和调用触发器

```
CREATE TRIGGER <触发器名>
<触发时机> <触发事件> ON <表名>
FOR EACH ROW
EXECUTE PROCEDURE <触发器的函数>;
```

示例：

【例 7】创建一个名为 TRI_PRO_STU 的触发器，在每次对学生成绩进行更新时，判断该学生成绩是否提升，如果提升，将该学生学号存入表

Progressive_Student(Sno)中；否则，将该学生学号存入表 Regressive_Student(Sno)中。

1.创建触发器函数

```
CREATE OR REPLACE FUNCTION tri_pro_stu_func()
RETURNS TRIGGER
AS $$
BEGIN
    IF NEW.Grade > OLD.Grade THEN INSERT INTO Progressive_Student (Sno)
VALUES (NEW.Sno);
    ELSE INSERT INTO Regressive_Student (Sno) VALUES (NEW.Sno);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

2.创建触发器，绑定到 SC 表的 Grade 字段更新事件

```
CREATE TRIGGER TRI_PRO_STU
AFTER UPDATE OF Grade ON SC
FOR EACH ROW
EXECUTE PROCEDURE tri_pro_stu_func();
```

7 删除触发器

```
DROP TRIGGER <触发器名>;
```

（需要指示删除哪个表的触发器，如 DROP TRIGGER my_trigger ON my_table;）

